

5. Driver Board Communication Protocol and Explanation

5.1 Servo Mode Control Modes and Explanation

Servo mode has 6 control modes:

Duty Cycle Mode: Specifies the motor's duty cycle voltage in a square wave driving form.

Current Loop Mode: Specifies the Iq current of the motor. As the motor output torque = Iq * Kt, it can be used as a torque loop.

Current Brake Mode: Specifies the braking current of the motor to fix the motor at the current position (pay attention to motor temperature during use).

Speed Mode: Specifies the running speed of the motor.

Position Mode: Specifies the position of the motor, and the motor will run to the specified position at the maximum speed.

Position-Speed Loop Mode: Specifies the position, speed, and acceleration of the motor. The motor will run to the specified position with the given acceleration and maximum speed.

The servo protocol uses the CAN protocol with an extended frame format. The format is as follows:

Can ID bits	[28]-[8]	[7]-[0]
Field name	Control mode	Source node ID

Control mode has values {0, 1, 2, 3, 4, 5, 6}, corresponding to 7 control modes.

Duty Cycle Mode: 0

Current Loop Mode: 1

Current Brake Mode: 2

Speed Mode: 3

Position Mode: 4

Set Origin Mode: 5

Position-Speed Loop Mode: 6

The following provides examples of controlling the motor in various modes:

The following are examples of invoking library functions and macro definitions:

```
typedef enum {
    CAN_PACKET_SET_DUTY = 0,          // Duty Cycle Mode
    CAN_PACKET_SET_CURRENT,          // Current Loop Mode
    CAN_PACKET_SET_CURRENT_BRAKE,    // Current Brake Mode
    CAN_PACKET_SET_RPM,              // Speed Mode
    CAN_PACKET_SET_POS,              // Position Mode
    CAN_PACKET_SET_ORIGIN_HERE,      // Set Origin Mode
    CAN_PACKET_SET_POS_SPD,          // Position-Speed Loop Mode
}
```

```

} CAN_PACKET_ID;

```

```

void comm_can_transmit_eid(uint32_t id, const uint8_t *data, uint8_t len) {
    uint8_t i=0;
    if (len > 8) {
        len = 8;
    }

    CanTxMsg TxMessage;
    TxMessage.StdId = 0;
    TxMessage.IDE = CAN_ID_EXT;
    TxMessage.ExtId = id;
    TxMessage.RTR = CAN_RTR_DATA;
    TxMessage.DLC = len;
    for(i=0;i<len;i++)
        TxMessage.Data[i]=data[i];
    CAN_Transmit(CHASSIS_CAN, &TxMessage); //CAN port sends TxMessage data
}

```

```

void buffer_append_int32(uint8_t* buffer, int32_t number, int32_t *index) {
    buffer[(*index)++] = number >> 24;
    buffer[(*index)++] = number >> 16;
    buffer[(*index)++] = number >> 8;
    buffer[(*index)++] = number;
}

```

```

void buffer_append_int16(uint8_t* buffer, int16_t number, int16_t *index) {
    buffer[(*index)++] = number >> 8;
    buffer[(*index)++] = number;
}

```

5.1.1 Duty Cycle Mode

Duty cycle mode data transmission definition

Data bits	Data[0]	Data[1]	Data[2]	Data[3]
Range	0~0xff	0~0xff	0~0xff	0~0xff
variables	Duty Cycle 25-32 bits	Duty Cycle 17-24 bits	Duty Cycle 9-16 bits	Duty Cycle 1-8 bits

```

void comm_can_set_duty(uint8_t controller_id, float duty) {
    int32_t send_index = 0;
    uint8_t buffer[4];
    buffer_append_int32(buffer, (int32_t)(duty * 100000.0), &send_index);
}

```

```

comm_can_transmit_eid(controller_id |
                        ((uint32_t)CAN_PACKET_SET_DUTY << 8), buffer, send_index);
}

```

5.1.2 Current Loop Mode

Current Loop mode data transmission definition

Data bits	Data[0]	Data[1]	Data[2]	Data[3]
Range	0~0xff	0~0xff	0~0xff	0~0xff
Variables	Current 25-32 bits	Current 17-24 bits	Current 9-16 bits	Current 1-8 bits

The current value is of type int32, and the value -60000-60000 represents -60-60A.

Example for Current Loop Mode Transmission:

```

void comm_can_set_current(uint8_t controller_id, float current) {
    int32_t send_index = 0;
    uint8_t buffer[4];
    buffer_append_int32(buffer, (int32_t)(current * 1000.0), &send_index);
    comm_can_transmit_eid(controller_id |
                        ((uint32_t)CAN_PACKET_SET_CURRENT << 8), buffer, send_index);
}

```

5.1.3 Current Brake Mode

Current Brake mode data transmission definition

Data bits	Data[0]	Data[1]	Data[2]	Data[3]
Range	0~0xff	0~0xff	0~0xff	0~0xff
Variables	Brake Current 25-32 bits	Brake Current 17-24 bits	Brake Current 9-16 bits	Brake Current 1-8 bits

The brake current value is of type int32, and the value 0-60000 represents 0-60A.

Example for Current Brake Mode Transmission:

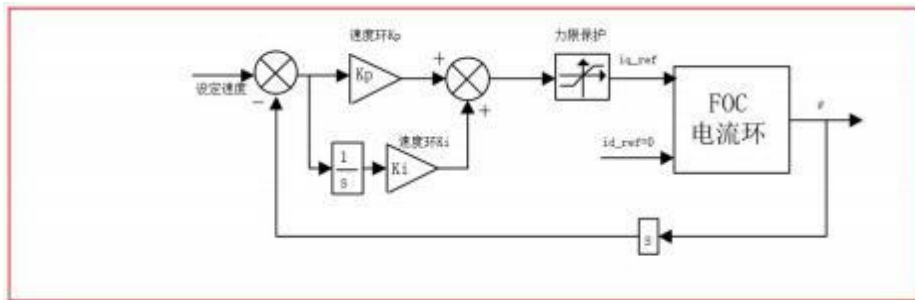
```

void comm_can_set_cb(uint8_t controller_id, float current) {
    int32_t send_index = 0;
    uint8_t buffer[4];
    buffer_append_int32(buffer, (int32_t)(current * 1000.0), &send_index);
    comm_can_transmit_eid(controller_id |
                        ((uint32_t)CAN_PACKET_SET_CURRENT_BRAKE << 8), buffer, send_index);
}

```

5.1.4 Speed Loop Mode

Speed loop simplified control block diagram



Speed Loop mode data transmission definition

Data bits	Data[0]	Data[1]	Data[2]	Data[3]
Range	0~0xff	0~0xff	0~0xff	0~0xff
Variables	Speed 25-32 bits	Speed 17-24 bits	Speed 9-16 bits	Speed 1-8 bits

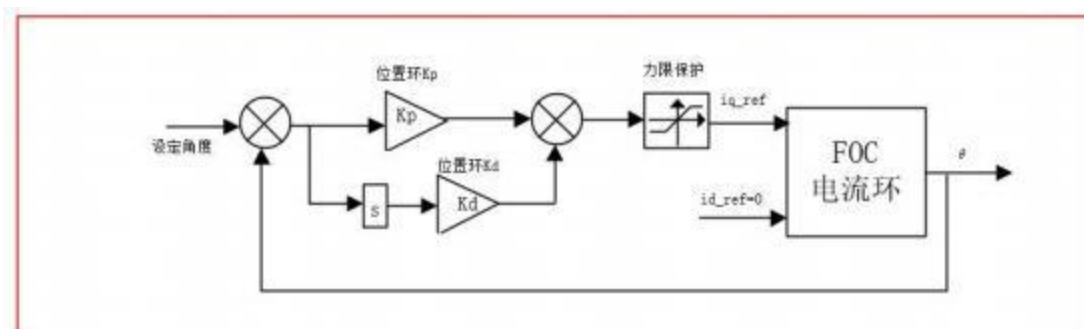
The speed value is of type int32, and the range is -100000-100000, representing -100000-100000 electrical RPM.

Example for Speed Loop Mode Transmission

```
void comm_can_set_rpm(uint8_t controller_id, float rpm) {
    int32_t send_index = 0;
    uint8_t buffer[4];
    buffer_append_int32(buffer, (int32_t)rpm, &send_index);
    comm_can_transmit_eid(controller_id |
        ((uint32_t)CAN_PACKET_SET_RPM << 8), buffer, send_index);
}
```

5.1.5 Position Loop Mode

Position loop simplified control block diagram



Position Loop mode data transmission definition

Data bits	Data[0]	Data[1]	Data[2]	Data[3]
Range	0~0xff	0~0xff	0~0xff	0~0xff
Variables	Position 25-32 bits	Position 17-24 bits	Position 9-16 bits	Position 1-8 bits

The position value is of type int32, and the range is -3600000000-3600000000, representing -36000°-36000°.

Example for Position Loop Mode Transmission:

```
void comm_can_set_pos(uint8_t controller_id, float pos) {
    int32_t send_index = 0;
    uint8_t buffer[4];
    buffer_append_int32(buffer, (int32_t)(pos * 10000.0), &send_index);
    comm_can_transmit_eid(controller_id |
        ((uint32_t)CAN_PACKET_SET_POS << 8), buffer, send_index);
}
```

5.1.6 Set Origin Mode

Data bits	Data[0]
Range	0~0x02
Variables	Set Instruction

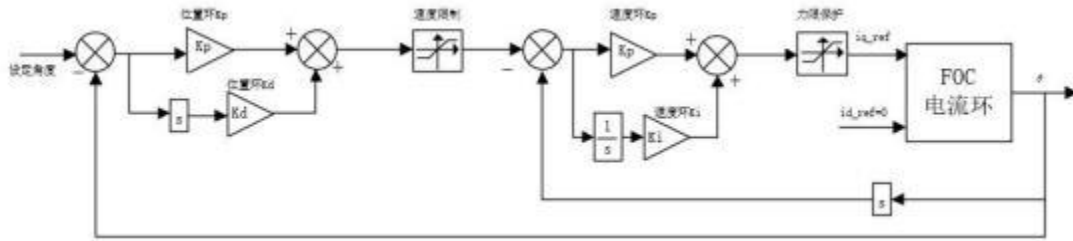
The setting command is of type uint8_t, where 0 represents setting a temporary origin (cleared after power-off), and 1 represents setting a permanent zero point (For dual encoder models only).

Example for Set Origin Mode Transmission

```
void comm_can_set_origin(uint8_t controller_id, uint8_t set_origin_mode) {
    int32_t send_index = 0;
    uint8_t buffer;
    buffer=set_origin_mode;
    comm_can_transmit_eid(controller_id |
        ((uint32_t) CAN_PACKET_SET_ORIGIN_HERE << 8), &buffer, send_index);
}
```

5.1.7 Position-Speed Loop Mode

Position-Speed loop simplified control block diagram



Position-Speed Loop mode data transmission definition

Data bits	Data[0]	Data[1]	Data[2]	Data[3]	Data[4]	Data[5]	Data[6]	Data[7]
Range	0~0xff	0~0xff	0~0xff	0~0xff	0~0xff	0~0xff	0~0xff	0~0xff
Variables	Position 25-32 bits	Position 17-24 bits	Position 9-16 bits	Position 1-8 bits	Speed High 8 bits	Speed Low 8 bits	Acceleration High 8 bits	Acceleration Low 8 bits

- Position: int32, range -360000000~360000000 representing -36000°~36000°.
- Speed: int16, range -32768~32767 representing -327680~327680 electrical RPM.
- Acceleration: int16, range 0~32767, representing 0~327670, 1 unit equals 10 electrical RPM/s².

```
void comm_can_set_pos_spd(uint8_t controller_id, float pos, int16_t spd, int16_t RPA ) {
    int32_t send_index = 0;
    int16_t send_index1 = 4;
    uint8_t buffer[8];
    buffer_append_int32(buffer, (int32_t)(pos * 10000.0), &send_index);
    buffer_append_int16(buffer, spd/ 10.0, & send_index1);
    buffer_append_int16(buffer, RPA/ 10.0, & send_index1);
    comm_can_transmit_eid(controller_id |
        ((uint32_t)CAN_PACKET_SET_POS_SPD << 8), buffer, send_index1);
}
```

5.2 Servo Mode Message Formats

5.2.1 Servo Mode CAN Upload Message Protocol

In servo mode, the motor CAN message uses periodic upload mode, and the upload frequency can be set to 1-500Hz, with an upload size of 8 bytes.

Data bits	Data[0]	Data[1]	Data[2]	Data[3]	Data[4]	Data[5]	Data[6]	Data[7]
Range	0~0xff	0~0xff	0~0xff	0~0xff	0~0xff	0~0xff	0~0xff	0~0xff
Variables	Position High 8 bits	Position Low 8 bits	Speed High 8 bits	Speed Low 8 bits	Current High 8 bits	Current Low 8 bits	Motor Temperature	Error Code

- **Position:** int16, range -32000~32000 represents -3200°~3200°.
- **Speed:** int16, range -32000~32000 represents -320000~320000 electrical RPM.
- **Current:** int16, range -6000~6000 represents -60~60A.
- **Motor Temperature:** int8, range -20~127 represents the driver board temperature -20°C~127°C.
- **Error Code:** uint8, 0 indicates no fault, 1 indicates motor over-temperature fault, 2 indicates over-current fault, 3 indicates over-voltage fault, 4 indicates under-voltage fault, 5 indicates encoder fault, 6 indicates MOSFET over-temperature fault, 7 indicates motor stall.

Example of Receiving Message:

```
void motor_receive(float* motor_pos,float*
motor_spd,float* cur,int_8* temp,int_8* error,rx_message)
{
    int16_t pos_int = (rx_message)->Data[0] << 8 | (rx_message)->Data[1]);
    int16_t spd_int = (rx_message)->Data[2] << 8 | (rx_message)->Data[3]);
    int16_t cur_int = (rx_message)->Data[4] << 8 | (rx_message)->Data[5]);
    &motor_pos= (float)( pos_int * 0.1f); // Motor Position
    &motor_spd= (float)( spd_int * 10.0f); // Motor Speed
    &motor_cur= (float) ( cur_int * 0.01f); // Motor Current
    &motor_temp= (rx_message)->Data[6] ; // Motor Temperature
    &motor_error= (rx_message)->Data[7] ; // Motor Error Code
}
```

5.2.2 Servo Mode Serial Message Protocol

The protocol for servo mode serial communication is as follows:

Frame (0x02)	Head	Data Length (excluding frame head, frame tail, and checksum)	Data Frame	Data	Checksum High 8 bits	Checksum Low 8 bits	Frame (0x03)	Tail
-----------------	------	--	------------	------	-------------------------	------------------------	-----------------	------

Checksum Bit Calculation Code Reference Chapter Five

Data Frame Definitions:

```
typedef enum {
  COMM_FW_VERSION = 0,
  COMM_JUMP_TO_BOOTLOADER,
  COMM_ERASE_NEW_APP,
  COMM_WRITE_NEW_APP_DATA,
  COMM_GET_VALUES,      // Get motor operating parameters
  COMM_SET_DUTY,        // Motor operates in duty cycle mode
  COMM_SET_CURRENT,     // Motor operates in current loop mode
  COMM_SET_CURRENT_BRAKE, // Motor operates in current brake mode
  COMM_SET_RPM,         // Motor operates in speed loop mode
  COMM_SET_POS,         // Motor operates in position loop mode
  COMM_SET_HANDBRAKE,   // Motor operates in handbrake current loop mode
  COMM_SET_DETECT,      // Motor real-time feedback current position command
  COMM_ROTOR_POSITION=22, // Motor feedback current position
  COMM_GET_VALUES_SETUP=50, // Motor single or multiple parameter acquisition command
  COMM_SET_POS_SPD=91,   // Motor operates in position-speed loop mode
  COMM_SET_POS_MULTI=92, // Set motor motion to single-turn mode
  COMM_SET_POS_SINGLE=93, // Set motor motion to multi-turn mode, range ±100 turns
  COMM_SET_POS_UNLIMITED=94, // Reserved
  COMM_SET_POS_ORIGIN=95, // Set motor origin
} COMM_PACKET_ID;
```


Motor feedback position command

Serial command: 02 02 0B 04 9C 7E 03 // Motor sends current position every 10 ms after receiving this command

Example of motor feedback position value transmission (Prior to this, send a feedback position command to the motor. After the motor receives it, it will send the current position every 10 milliseconds.)

Serial command: 02 05 16 00 1A B6 64 D5 F4 03

Pos=(float)buffer_get_int32(data, &ind) / 1000.0

Motor single or multiple parameter acquisition command example

Serial command: 02 05 32 00 00 00 01 58 4C 03 // Get motor temperature command

Instruction Explanation: This command allows the retrieval of single or multiple motor parameters. The parameters to be retrieved are determined by the 4-byte data segment. Corresponding to a bit being set to 1, the motor will return the corresponding motor parameter, and for a bit set to 0, that field will be excluded.

The motor parameters corresponding to each bit are as follows:

Bit 32-19	Bit 18	Bit 17	Bit 16	Bit 10-15	Bit 9	Bit 8	Bit 7
Reserved	Motor ID (1 byte)	Motor position (4 bytes)	Motor error flag (1 byte)	Reserved	Input voltage (2 bytes)	Motor speed (4 bytes)	Duty cycle (2 bytes)
Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1		
Iq current (4 bytes)	Id current (4 bytes)	Input current (4 bytes)	Output current (4 bytes)	Motor temperature (2 bytes)	MOS temperature (2 bytes)		

After receiving this command, the motor will respond with the corresponding parameters.

Example: 02 03 32 00 81 2A 6C 03 // Feedback motor temperature

Conversion formulas for parameters sent by the motor:

MOS temperature = (float)buffer_get_int16(data, &ind) / 10.0;

Motor temperature = (float)buffer_get_int16(data, &ind) / 10.0;

Output current = (float)buffer_get_int32(data, &ind) / 100.0;

Input current = (float)buffer_get_int32(data, &ind) / 100.0;

Throttle Value = (float)buffer_get_int16(data, &ind) / 1000.0;

Motor Speed = (float)buffer_get_int32(data, &ind);

Input voltage = (float)buffer_get_int16(data, &ind) / 10.0;

Motor position = (float)buffer_get_int32(data, &ind) / 1000000.0;

Motor ID number = data;

Motor error status code:

```
typedef enum {
    FAULT_CODE_NONE = 0,
    FAULT_CODE_OVER_VOLTAGE,// Overvoltage
    FAULT_CODE_UNDER_VOLTAGE,// Undervoltage
    FAULT_CODE_DRV,// Driver fault
    FAULT_CODE_ABS_OVER_CURRENT,// Motor overcurrent
    FAULT_CODE_OVER_TEMP_FET,// MOS overtemperature
    FAULT_CODE_OVER_TEMP_MOTOR,// Motor overtemperature
    FAULT_CODE_GATE_DRIVER_OVER_VOLTAGE,// Driver overvoltage
    FAULT_CODE_GATE_DRIVER_UNDER_VOLTAGE,// Driver undervoltage
    FAULT_CODE_MCU_UNDER_VOLTAGE,// MCU undervoltage
    FAULT_CODE_BOOTING_FROM_WATCHDOG_RESET,// Undervoltage
    FAULT_CODE_ENCODER_SPI,// SPI encoder fault
    FAULT_CODE_ENCODER_SINCOS_BELOW_MIN_AMPLITUDE,// Encoder below minimum
    amplitude

    FAULT_CODE_ENCODER_SINCOS_ABOVE_MAX_AMPLITUDE,//Encoder above maximum
    amplitude

    FAULT_CODE_FLASH_CORRUPTION,// Flash fault
    FAULT_CODE_HIGH_OFFSET_CURRENT_SENSOR_1,// Current sampling channel 1 fault
    FAULT_CODE_HIGH_OFFSET_CURRENT_SENSOR_2,// Current sampling channel 2 fault
    FAULT_CODE_HIGH_OFFSET_CURRENT_SENSOR_3,// Current sampling channel 3 fault
    FAULT_CODE_UNBALANCED_CURRENTS,// Unbalanced currents
} mc_fault_code;
```

II. Control command examples:

Example of duty cycle mode transmission

```
Serial command: 02 05 05 00 00 4E 20 29 F6 03 // 0.20 duty cycle
Serial command: 02 05 05 FF FF B1 E0 77 85 03 // -0.20 duty cycle
Duty=(float)buffer_get_int32(data, &ind) / 100000.0 //Value as the received 4-byte
data/10000.0
```

Example of current loop transmission

```
Serial command: 02 05 06 00 00 13 88 8B 25 03 // 5 A IQ current
Serial command: 02 05 06 FF FF EC 78 E3 05 03 // - 5 A IQ current
Current=(float)buffer_get_int32(data, &ind) / 1000.0 //Value as the received 4-byte
data/1000.0
```

Example of brake current mode transmission

Serial command: 02 05 07 00 00 13 88 21 74 03 // 5A brake current
 Serial command: 02 05 07 FF FF EC 78 49 54 03 // - 5A brake current
 I_Brake=(float)buffer_get_int32(data, &ind) / 1000.0 //Value as the received 4-byte data/1000.0

Example of speed loop transmission

Serial command: 02 05 08 00 00 03 E8 2B 58 03 // 1000 ERPM electrical speed
 Serial command: 02 05 08 FF FF FC 18 43 78 03 // - 1000 ERPM electrical speed
 Speed=(float)buffer_get_int32(data, &ind) //Value as the received 4-byte data

Example of position loop transmission

Serial command: 02 05 09 0A BA 95 00 1E E7 03 // Motor rotates to 180 degrees
 Serial command: 02 05 09 05 5D 4A 80 7B 29 03 // Motor rotates to 90 degrees
 Pos=(float)buffer_get_int32(data, &ind) / 1000000.0 //Value as the received 4-byte data/1000000.0

Example of handbrake current mode transmission

Serial command: 02 05 0A 00 00 13 88 00 0E 03 // 5A handbrake current electrical speed
 Serial command: 02 05 0A FF FF EC 78 68 2E 03 // 5A handbrake current electrical speed
 HAND_Brake=(float)buffer_get_int32(data, &ind) / 1000.0 // Value as the received 4-byte data/1000.0

Example of position-speed loop mode transmission

Serial command: 02 0D 5B 00 02 BF 20 00 00 13 88 00 00 75 30 A5 AC 03
 /*
 180 degrees, speed 5000 ERPM, acceleration 30000/S
 Data segment: Position + Speed + Acceleration
 */
 Pos=(float)buffer_get_int32(data, &ind) / 1000.0 // Position value as the received 4-byte data/1000.0
 Speed=(float)buffer_get_int32(data, &ind) // Value as the received 4-byte data
 Acc_Speed=(float)buffer_get_int32(data, &ind) // Value as the received 4-byte data

Example of multi mode transmission

Serial command: 02 05 5C 00 00 00 00 9E 19 03 // Set motor position loop for multi-turn operation ± 100 turns

Example of single mode transmission

Serial command: 02 05 5D 00 00 00 00 34 48 03 // Set motor position loop for single-turn operation 0-360 degrees

Example of setting the current position as zero position transmission

Serial command: 02 02 5F 01 0E A0 03 //Set motor current position loop as position loop zero reference point

Shortest distance return-to-zero command

Serial command: 02 05 65 00 00 00 00 3A 8B 03 // **Make the motor return to the relative zero position in the shortest distance**

Serial checksum:

```
unsigned short crc16(unsigned char *buf, unsigned int len) {
    unsigned int i;
    unsigned short cksum = 0;
    for (i = 0; i < len; i++) {
        cksum = crc16_tab[(((cksum >> 8) ^ *buf++) & 0xFF)] ^ (cksum << 8);
    }
    return cksum;
}

const unsigned short crc16_tab[] = { 0x0000, 0x1021, 0x2042, 0x3063, 0x4084,
0x50a5, 0x60c6, 0x70e7, 0x8108, 0x9129, 0xa14a, 0xb16b, 0xc18c, 0xd1ad,
0xe1ce, 0xf1ef, 0x1231, 0x0210, 0x3273, 0x2252, 0x52b5, 0x4294, 0x72f7,
0x62d6, 0x9339, 0x8318, 0xb37b, 0xa35a, 0xd3bd, 0xc39c, 0xf3ff, 0xe3de,
0x2462, 0x3443, 0x0420, 0x1401, 0x64e6, 0x74c7, 0x44a4, 0x5485, 0xa56a,
0xb54b, 0x8528, 0x9509, 0xe5ee, 0xf5cf, 0xc5ac, 0xd58d, 0x6553, 0x7572,
0x1611, 0x0630, 0x76d7, 0x66f6, 0x5695, 0x46b4, 0xb75b, 0xa77a, 0x9719,
0x8738, 0xf7df, 0xe7fe, 0xd79d, 0xc7bc, 0x48c4, 0x58e5, 0x6886, 0x78a7,
0x0840, 0x1861, 0x2802, 0x3823, 0xc9cc, 0xd9ed, 0xe98e, 0xf9af, 0x8948,
0x9969, 0xa90a, 0xb92b, 0x5af5, 0x4ad4, 0x7ab7, 0x6a96, 0x1a71, 0x0a50,
0x3a33, 0x2a12, 0xdbfd, 0xcbbdc, 0xfbbf, 0xeb9e, 0x9b79, 0x8b58, 0xbb3b,
0xab1a, 0x6ca6, 0x7c87, 0x4ce4, 0x5cc5, 0x2c22, 0x3c03, 0x0c60, 0x1c41,
0xedae, 0xfd8f, 0xcdec, 0xddcd, 0xad2a, 0xbd0b, 0x8d68, 0x9d49, 0x7e97,
0x6eb6, 0x5ed5, 0x4ef4, 0x3e13, 0x2e32, 0x1e51, 0x0e70, 0xff9f, 0xefbe,
0xdfdd, 0xcffc, 0xbf1b, 0xaf3a, 0x9f59, 0x8f78, 0x9188, 0x81a9, 0xb1ca,
0xa1eb, 0xd10c, 0xc12d, 0xf14e, 0xe16f, 0x1080, 0x00a1, 0x30c2, 0x20e3,
0x5004, 0x4025, 0x7046, 0x6067, 0x83b9, 0x9398, 0xa3fb, 0xb3da, 0xc33d,
0xd31c, 0xe37f, 0xf35e, 0x02b1, 0x1290, 0x22f3, 0x32d2, 0x4235, 0x5214,
0x6277, 0x7256, 0xb5ea, 0xa5cb, 0x95a8, 0x8589, 0xf56e, 0xe54f, 0xd52c,
0xc50d, 0x34e2, 0x24c3, 0x14a0, 0x0481, 0x7466, 0x6447, 0x5424, 0x4405,
0xa7db, 0xb7fa, 0x8799, 0x97b8, 0xe75f, 0xf77e, 0xc71d, 0xd73c, 0x26d3,
0x36f2, 0x0691, 0x16b0, 0x6657, 0x7676, 0x4615, 0x5634, 0xd94c, 0xc96d,
0xf90e, 0xe92f, 0x99c8, 0x89e9, 0xb98a, 0xa9ab, 0x5844, 0x4865, 0x7806,
```

```
0x6827, 0x18c0, 0x08e1, 0x3882, 0x28a3, 0xcb7d, 0xdb5c, 0xeb3f, 0xfb1e,  
0x8bf9, 0x9bd8, 0abbb, 0xbb9a, 0x4a75, 0x5a54, 0x6a37, 0x7a16, 0x0af1,  
0x1ad0, 0x2ab3, 0x3a92, 0xfd2e, 0xed0f, 0xdd6c, 0xcd4d, 0xbdaa, 0xad8b,  
0x9de8, 0x8dc9, 0x7c26, 0x6c07, 0x5c64, 0x4c45, 0x3ca2, 0x2c83, 0x1ce0,  
0x0cc1, 0xef1f, 0xff3e, 0xcf5d, 0xdf7c, 0xaf9b, 0xbfba, 0x8fd9, 0x9ff8,  
0x6e17, 0x7e36, 0x4e55, 0x5e74, 0x2e93, 0x3eb2, 0x0ed1, 0x1ef0 };
```

```
//Arrange int16 data
```

```
void buffer_append_int16(uint8_t* buffer, int16_t number, int32_t *index) {  
    buffer[(*index)++] = number >> 8;  
    buffer[(*index)++] = number;  
}
```

```
//Arrange uint16 data
```

```
void buffer_append_uint16(uint8_t* buffer, uint16_t number, int32_t *index) {  
    buffer[(*index)++] = number >> 8;  
    buffer[(*index)++] = number;  
}
```

```
//Arrange int32 data
```

```
void buffer_append_int32(uint8_t* buffer, int32_t number, int32_t *index) {  
    buffer[(*index)++] = number >> 24;  
    buffer[(*index)++] = number >> 16;  
    buffer[(*index)++] = number >> 8;  
    buffer[(*index)++] = number;  
}
```

```
//Arrange uint32 data
```

```
void buffer_append_uint32(uint8_t* buffer, uint32_t number, int32_t *index) {  
    buffer[(*index)++] = number >> 24;  
    buffer[(*index)++] = number >> 16;  
    buffer[(*index)++] = number >> 8;  
    buffer[(*index)++] = number;  
}
```

```
//Arrange int64 data
```

```
void buffer_append_int64(uint8_t* buffer, int64_t number, int32_t *index) {  
    buffer[(*index)++] = number >> 56;  
    buffer[(*index)++] = number >> 48;  
    buffer[(*index)++] = number >> 40;  
    buffer[(*index)++] = number >> 32;  
    buffer[(*index)++] = number >> 24;
```

```
        buffer[( *index)++] = number >> 16;
        buffer[( *index)++] = number >> 8;
        buffer[( *index)++] = number;
    }

//Arrange uint64 data
void buffer_append_uint64(uint8_t* buffer, uint64_t number, int32_t *index) {
    buffer[( *index)++] = number >> 56;
    buffer[( *index)++] = number >> 48;
    buffer[( *index)++] = number >> 40;
    buffer[( *index)++] = number >> 32;
    buffer[( *index)++] = number >> 24;
    buffer[( *index)++] = number >> 16;
    buffer[( *index)++] = number >> 8;
    buffer[( *index)++] = number;
}

//CRC Checksum
unsigned short crc16(unsigned char *buf, unsigned int len) {
    unsigned int i;
    unsigned short cksum = 0;
    for (i = 0; i < len; i++) {
        cksum = crc16_tab[(((cksum >> 8) ^ *buf++) & 0xFF)] ^ (cksum << 8);
    }
    return cksum;
}

//Organize and send data packet
void packet_send_packet(unsigned char *data, unsigned int len, int handler_num) {
    int b_ind = 0;
    unsigned short crc;
    if (len > PACKET_MAX_PL_LEN) {
        return;
    }
    if (len <= 256) {
        handler_states[handler_num].tx_buffer[b_ind++] = 2;
        handler_states[handler_num].tx_buffer[b_ind++] = len;
    } else {
        handler_states[handler_num].tx_buffer[b_ind++] = 3;
        handler_states[handler_num].tx_buffer[b_ind++] = len >> 8;
        handler_states[handler_num].tx_buffer[b_ind++] = len & 0xFF;
    }
}
```

```

memcpy(handler_states[handler_num].tx_buffer + b_ind, data, len);
b_ind += len;

crc = crc16(data, len);
handler_states[handler_num].tx_buffer[b_ind++] = (uint8_t)(crc >> 8);
handler_states[handler_num].tx_buffer[b_ind++] = (uint8_t)(crc & 0xFF);
handler_states[handler_num].tx_buffer[b_ind++] = 3;

if (handler_states[handler_num].send_func) {
    handler_states[handler_num].send_func(handler_states[handler_num].tx_buffer,
b_ind);
}
}

```

5.3 MIT Mode Communication Protocol

Special CAN Codes

Enter Motor Control Mode: {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFC }

Exit Motor Control Mode: {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFD }

Set Current Motor Position as zero position: {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFE }

Note: It is necessary to enter Motor Control Mode before controlling the motor using CAN communication!

PS: (If you want to read the current state in a stateless manner, the command to send is {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFC })

MIT Mode Driver Board Receive Data Definition

Identifier: Set Motor ID (default is 1)

Frame Type: Standard Frame

Frame Format: DATA

Data Length Code (DLC): 8 Bytes

Data Field	DATA[0]	DATA[1]	DATA[2]	DATA[3]	
Data Bits	7-0	7-0	7-0	7-4	3-0
Data Content	Motor Position High 8 bits	Motor Position Low 8 bits	Motor Speed High 8 bits	Motor Speed Low 4 bits	KP Value High 4 bits

Data Field	DATA[4]	DATA[5]	DATA[6]	DATA[7]	
Data Bits	7-0	7-0	7-4	3-0	0-7
Data Content	KP Value Low 8 bits	KD Value High 8 bits	KD Value Low 4 bits	Current Value High 4 bits	Current Value Low 8 bits